```python
# MelSpectrogram_from_scratch.py

import numpy as np
import matplotlib.pyplot as plt

def hz_to_mel(hz):
    """Convert Frequency in Hz to Mel scale."""
    return 2595 * np.log10(1 + hz / 700.0)

def mel_to_hz(mel):
    """Convert Mel scale back to Frequency in Hz."""
    return 700 * (10**(mel / 2595.0) - 1)

def create_mel_filterbank(sr, n_fft, n_mels=128, fmin=0, fmax=None):
    """Build a triangular Mel-filterbank matrix from scratch."""
    if fmax is None:
        fmax = sr / 2 # Nyquist frequency

    # 1. Map min/max Hz frequencies to Mel scale
    mel_min = hz_to_mel(fmin)
    mel_max = hz_to_mel(fmax)

    # 2. Generate linearly spaced points in the Mel scale (plus 2 for the edges)
    mel_points = np.linspace(mel_min, mel_max, n_mels + 2)

    # 3. Convert Mel points back to Hz
    hz_points = mel_to_hz(mel_points)

    # 4. Map Hz points to FFT bin numbers
    bins = np.floor((n_fft + 1) * hz_points / sr).astype(int)

    # 5. Construct the triangular filter banks
    num_fft_bins = int(n_fft // 2 + 1)
    fbank = np.zeros((n_mels, num_fft_bins))

    for m in range(1, n_mels + 1):
        f_m_minus = bins[m - 1]   # Left edge
        f_m = bins[m]             # Center peak
        f_m_plus = bins[m + 1]    # Right edge

        for k in range(f_m_minus, f_m):
            fbank[m - 1, k] = (k - bins[m - 1]) / (bins[m] - bins[m - 1])
        for k in range(f_m, f_m_plus):
            fbank[m - 1, k] = (bins[m + 1] - k) / (bins[m + 1] - bins[m])

    return fbank

def stft_from_scratch(signal, frame_size, hop_size, n_fft):
    """Compute the Short-Time Fourier Transform using overlapping windows."""
    # Apply standard Hann window to smooth block edges
    window = np.hanning(frame_size)

    # Pad signal so frames center properly
    pad_amount = int(n_fft // 2)
    signal = np.pad(signal, pad_amount, mode='reflect')

    # Slice the signal into overlapping frames
    num_frames = int((len(signal) - frame_size) // hop_size) + 1
    num_fft_bins = int(n_fft // 2 + 1)
    stft_matrix = np.zeros((num_fft_bins, num_frames), dtype=np.complex128)

    for t in range(num_frames):
        start = t * hop_size
        end = start + frame_size

```

```
65          frame = signal[start:end] * window
66
67          # Compute discrete Fast Fourier Transform (FFT)
68          fft_complex = np.fft.fft(frame, n=n_fft)
69          # Keep only positive frequencies up to the Nyquist ceiling
70          stft_matrix[:, t] = fft_complex[:num_fft_bins]
71
72      return stft_matrix
73
74  def mel_spectrogram_from_scratch(signal, sr, frame_size=1024, hop_size=512, n_fft=1024,
    n_mels=128):
75      """Pipeline to convert raw audio waveform into a log-Mel Spectrogram."""
76      # Step 1: Compute STFT
77      stft_matrix = stft_from_scratch(signal, frame_size, hop_size, n_fft)
78
79      # Step 2: Convert complex values to Power Spectrum
80      power_spectrogram = np.abs(stft_matrix) ** 2
81
82      # Step 3: Compute Mel Filter bank
83      mel_fbank = create_mel_filterbank(sr, n_fft, n_mels=n_mels)
84
85      # Step 4: Dot product maps linear frequency bins to Mel bins
86      mel_spectrogram = np.dot(mel_fbank, power_spectrogram)
87
88      # Step 5: Convert to Log scale (Decibels) to match human perception
89      # Add a tiny epsilon to prevent log(0) domain errors
90      log_mel_spectrogram = 10 * np.log10(mel_spectrogram + 1e-10)
91
92      return log_mel_spectrogram
93
94  # --- Verification & Demo ---
95  if __name__ == "__main__":
96      # Generate 3 seconds of a artificial synthetic chirp signal (sweeping frequency)
97      sr = 22050
98      duration = 3.0
99      t = np.linspace(0, duration, int(sr * duration), endpoint=False)
100     # Exponential chirp from 100Hz to 8000Hz
101     signal = np.sin(2 * np.pi * 100 * (8000 / 100) ** (t / duration))
102
103     # Generate the custom mel spectrogram
104     mel_spec = mel_spectrogram_from_scratch(signal, sr=sr, n_mels=128)
105
106     # Display the result
107     plt.figure(figsize=(10, 4))
108     plt.imshow(mel_spec, aspect='auto', origin='lower', cmap='viridis')
109     plt.title('Custom Mel Spectrogram From Scratch')
110     plt.ylabel('Mel Bin Index')
111     plt.xlabel('Time Frame Index')
112     plt.colorbar(label='Decibels (dB)')
113     plt.tight_layout()
114     plt.show()
115
```